Day 21 - SQL 기본: DDL과 DML

1. 데이터베이스란?

왜 필요한가?

여러분이 만든 웹 애플리케이션에서 사용자가 회원가입을 하고, 게시글을 작성하고, 댓글을 다는 상황을 생각해 보세요. 이 모든 데이터를 어디에 저장할까요? 파일에 저장할 수도 있지만, 데이터가 많아지면 문제가 생깁니다. **데이터베이스(Database, DB)**는 데이터를 체계적으로 저장하고, 빠르게 검색하고, 안전하게 관리하기 위한 전문 시스템입니다.

실생활 비유: 체계적인 서류 캐비넷

데이터베이스를 **거대한 서류 캐비넷**이라고 생각하세요.

- **서류 캐비넷 전체** = 데이터베이스
- **캐비넷의 각 서랍** = 테이블 (Table)
- **서랍 안의 서류 한 장** = 행/레코드 (Row/Record)
- **서류의 각 항목 (이름, 전화번호, 주소)** = 열/컬럼 (Column)

일반 파일에 데이터를 저장하는 것은 마치 서류를 책상 위에 아무렇게나 쌓아두는 것과 같습니다. 처음에는 괜찮지만, 서류가 수천 장이 되면 원하는 서류를 찾기가 매우 어려워집니다.

왜 파일 대신 DB를 쓰는가?

비교 항목	파일 저장	데이터베이스
검색 속도	전체를 읽어야 함 (느림)	인덱스로 빠르게 검색
동시 접근	파일 충돌 위험	여러 사용자가 동시에 접근 가능
데이터 무결성	보장 어려움	제약조건으로 데이터 품질 보장
백업/복구	수동으로 해야 함	자동 백업, 트랜잭션 복구
관계 표현	매우 복잡	JOIN으로 간단하게

```
// 파일로 저장하면? (users.json)
[
  { "id": 1, "name": "김철수", "email": "chulsoo@email.com" },
  { "id": 2, "name": "이영희", "email": "younghee@email.com" },
  // ... 10만 건이면? 파일 전체를 읽어야 검색 가능!
]

// 데이터베이스라면?
// SELECT * FROM users WHERE name = '김철수';
// → 인덱스를 이용해 즉시 검색!
```

관계형 데이터베이스 (RDBMS)

관계형 데이터베이스는 데이터를 **표(Table)** 형태로 저장합니다. 엑셀 스프레드시트를 떠올리면 이해하기 쉽습니다.

```
┌──────────────────────────────────────────┐
│              users 테이블                 │
├────┬──────────┬──────────────────┬────────┤
│ id │ name     │ email            │ age    │
├────┼──────────┼──────────────────┼────────┤
│ 1  │ 김철수   │ chulsoo@email.com │ 25     │
│ 2  │ 이영희   │ younghee@email.com │ 30     │
│ 3  │ 박민수   │ minsoo@email.com  │ 28     │
└────┴──────────┴──────────────────┴────────┘
```

- **행(Row)**: 하나의 데이터 (예: 김철수라는 한 명의 사용자)
- **열(Column)**: 데이터의 속성 (예: 이름, 이메일, 나이)

---

2. SQLite vs PostgreSQL vs MySQL

SQL(Structured Query Language)은 데이터베이스를 다루는 **표준 언어**입니다. 다양한 데이터베이스 시스템이 있지만, 기본 SQL 문법은 거의 동일합니다.

특징	SQLite	PostgreSQL	MySQL
설치	설치 불필요	서버 설치 필요	서버 설치 필요
파일	파일 하나 (.db)	서버 프로세스	서버 프로세스
적합한 용도	학습, 소규모 앱, 모바일	대규모 앱, 복잡한 쿼리	웹 서비스, 범용
동시 접속	제한적	우수	우수
가격	무료	무료	무료 (Community)

SQLite: 파일 하나로 동작하는 가벼운 DB

이 과정에서는 **SQLite**를 주로 사용합니다. SQLite는 별도의 서버 설치 없이 **파일 하나**로 동작하는 가벼운 데이터베이스입니다.

- 스마트폰의 카카오톡, 연락처 앱도 내부적으로 SQLite를 사용합니다
- 학습용으로 완벽하며, 소규모 프로젝트에도 충분합니다
- SQL 문법을 익히면 PostgreSQL, MySQL로 쉽게 전환할 수 있습니다

---

3. DDL: 테이블 만들기

DDL(Data Definition Language)은 데이터베이스의 **구조를 정의**하는 SQL 명령어입니다. 건물을 짓기 전에 설계도를 그리는 것과 같습니다.

CREATE TABLE: 테이블 생성

```sql
-- 기본 형태
CREATE TABLE 테이블이름 (
  컬럼이름1 자료형 제약조건,
  컬럼이름2 자료형 제약조건,
  ...
);

-- 사용자 테이블 만들기
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  age INTEGER DEFAULT 0,
  created_at TEXT DEFAULT (datetime('now'))
);
```

위 SQL을 줄별로 해석해 보겠습니다:

줄	의미
`id INTEGER PRIMARY KEY AUTOINCREMENT`	고유 번호, 자동 증가
`name TEXT NOT NULL`	이름, 빈 값 불가
`email TEXT NOT NULL UNIQUE`	이메일, 빈 값 불가 + 중복 불가
`age INTEGER DEFAULT 0`	나이, 미입력 시 0
`created_at TEXT DEFAULT (datetime('now'))`	생성 시각, 자동 기록

주요 제약조건 (Constraints)

```sql
-- PRIMARY KEY: 각 행을 고유하게 식별하는 키
-- 비유: 학생의 학번, 주민등록번호
id INTEGER PRIMARY KEY

-- AUTOINCREMENT: 새 행이 추가될 때마다 자동으로 번호가 1씩 증가
-- 비유: 번호표 발급기
id INTEGER PRIMARY KEY AUTOINCREMENT

-- NOT NULL: 반드시 값이 있어야 함 (빈 값 불허)
-- 비유: 필수 입력 항목 (*)
name TEXT NOT NULL

-- UNIQUE: 중복된 값이 들어갈 수 없음
-- 비유: 아이디는 한 사람만 사용 가능
email TEXT UNIQUE

-- DEFAULT: 값을 넣지 않으면 기본값 사용
-- 비유: 설문조사에서 "선택 안 함"이 기본
age INTEGER DEFAULT 0

-- CHECK: 조건을 만족하는 값만 허용
-- 비유: 나이는 0 이상만 가능
age INTEGER CHECK(age ≥ 0)
```

DROP TABLE: 테이블 삭제

```sql
-- 테이블 삭제 (주의! 데이터도 모두 삭제됩니다)
DROP TABLE users;

-- 안전한 삭제 (테이블이 있을 때만 삭제)
DROP TABLE IF EXISTS users;
```

ALTER TABLE: 테이블 수정

```sql
-- 컬럼 추가
ALTER TABLE users ADD COLUMN phone TEXT;

-- 테이블 이름 변경
ALTER TABLE users RENAME TO members;

-- 컬럼 이름 변경 (SQLite 3.25.0 이상)
ALTER TABLE users RENAME COLUMN phone TO phone_number;
```

자주 하는 실수

1. **테이블 이름에 예약어 사용**: `order`, `group`, `table` 등은 SQL 예약어이므로 테이블 이름으로 사용하면 오류가 납니다. `orders`, `user_groups`처럼 변경하세요.
2. **PRIMARY KEY 없이 테이블 생성**: 고유하게 행을 식별할 수 없어서 나중에 수정, 삭제가 어려워집니다.
3. **NOT NULL을 빠뜨리는 경우**: 중요한 필드에 빈 값이 들어가 데이터 품질이 떨어집니다.

---

4. DML: 데이터 다루기

DML(Data Manipulation Language)은 데이터를 **추가, 조회, 수정, 삭제**하는 SQL 명령어입니다. 흔히 **CRUD**라고 부릅니다:

- **C**reate (생성) → INSERT
- **R**ead (조회) → SELECT
- **U**pdate (수정) → UPDATE
- **D**elete (삭제) → DELETE

INSERT INTO: 데이터 넣기

```sql
-- 기본 형태
INSERT INTO 테이블이름 (컬럼1, 컬럼2, ...) VALUES (값1, 값2, ...);

-- 사용자 추가
INSERT INTO users (name, email, age) VALUES ('김철수', 'chulsoo@email.com', 25);
INSERT INTO users (name, email, age) VALUES ('이영희', 'younghee@email.com', 30);
INSERT INTO users (name, email, age) VALUES ('박민수', 'minsoo@email.com', 28);

-- 여러 행을 한 번에 추가
INSERT INTO users (name, email, age) VALUES
  ('최지은', 'jieun@email.com', 22),
  ('정우성', 'woosung@email.com', 35),
  ('한소희', 'sohee@email.com', 27);
```

SELECT: 데이터 꺼내기

SELECT는 가장 많이 사용하는 SQL 명령어입니다.

```sql
-- 모든 데이터 조회
SELECT * FROM users;

-- 특정 컬럼만 조회
SELECT name, email FROM users;

-- 별칭(alias) 사용
SELECT name AS 이름, email AS 이메일 FROM users;
```

WHERE 조건절

```sql
-- 나이가 25인 사용자
SELECT * FROM users WHERE age = 25;

-- 나이가 25 이상인 사용자
SELECT * FROM users WHERE age ≥ 25;

-- 이름이 '김철수'인 사용자
SELECT * FROM users WHERE name = '김철수';

-- 여러 조건 (AND: 둘 다 만족)
SELECT * FROM users WHERE age ≥ 25 AND age ≤ 30;

-- 여러 조건 (OR: 하나라도 만족)
SELECT * FROM users WHERE name = '김철수' OR name = '이영희';

-- IN: 여러 값 중 하나와 일치
SELECT * FROM users WHERE age IN (25, 28, 30);

-- LIKE: 패턴 매칭
SELECT * FROM users WHERE name LIKE '김%';    -- '김'으로 시작
SELECT * FROM users WHERE email LIKE '%@email.com';  -- '@email.com'으로 끝남
SELECT * FROM users WHERE name LIKE '_철_';    -- 가운데가 '철'인 3글자

-- BETWEEN: 범위
SELECT * FROM users WHERE age BETWEEN 25 AND 30;

-- IS NULL / IS NOT NULL
SELECT * FROM users WHERE phone IS NULL;
SELECT * FROM users WHERE phone IS NOT NULL;
```

ORDER BY 정렬

```sql
-- 나이 오름차순 (작은 것부터)
SELECT * FROM users ORDER BY age ASC;

-- 나이 내림차순 (큰 것부터)
SELECT * FROM users ORDER BY age DESC;

-- 여러 기준으로 정렬 (나이 오름차순, 같으면 이름 오름차순)
SELECT * FROM users ORDER BY age ASC, name ASC;
```

LIMIT 제한

```sql
-- 상위 5개만 조회
SELECT * FROM users LIMIT 5;

-- 6번째부터 5개 조회 (페이징에 활용)
SELECT * FROM users LIMIT 5 OFFSET 5;

-- 나이가 가장 많은 3명
SELECT * FROM users ORDER BY age DESC LIMIT 3;
```

UPDATE: 데이터 수정

```sql
-- 기본 형태
UPDATE 테이블이름 SET 컬럼 = 값 WHERE 조건;

-- 김철수의 나이를 26으로 변경
UPDATE users SET age = 26 WHERE name = '김철수';

-- 여러 컬럼 동시 수정
UPDATE users SET age = 26, email = 'new@email.com' WHERE id = 1;
```

주의! WHERE 절을 빼먹으면 **모든 행**이 수정됩니다!

```sql
-- 이렇게 하면 모든 사용자의 나이가 26으로 바뀝니다!
UPDATE users SET age = 26;  -- WHERE 절이 없음! 위험!
```

DELETE: 데이터 삭제

```sql
-- 기본 형태
DELETE FROM 테이블이름 WHERE 조건;

-- id가 3인 사용자 삭제
DELETE FROM users WHERE id = 3;

-- 나이가 30 이상인 사용자 삭제
DELETE FROM users WHERE age ≥ 30;
```

주의! UPDATE와 마찬가지로 WHERE 절을 빼먹으면 **모든 행**이 삭제됩니다!

```sql
-- 이렇게 하면 모든 데이터가 삭제됩니다!
DELETE FROM users;  -- WHERE 절이 없음! 매우 위험!
```

자주 하는 실수

1. **UPDATE/DELETE에서 WHERE 절 빠뜨리기**: 가장 위험한 실수입니다. 항상 WHERE 절을 먼저 쓰는 습관을 들이세요.
2. **문자열에 작은따옴표 사용 안 함**: SQL에서 문자열은 반드시 작은따옴표(`'`)로 감싸야 합니다. 큰따옴표(`"`)는 컬럼/테이블 이름에 사용합니다.
3. **SELECT 결과를 확인 안 하고 UPDATE/DELETE 실행**: 먼저 SELECT로 대상을 확인한 후 UPDATE/DELETE를 실행하세요.

```sql
-- 좋은 습관: 먼저 SELECT로 확인
SELECT * FROM users WHERE age ≥ 30;
-- 결과를 확인한 후
DELETE FROM users WHERE age ≥ 30;
```

---

5. SQLite 자료형

SQLite는 다른 DB에 비해 자료형이 단순합니다.

자료형	설명	예시
INTEGER	정수	1, 42, -10
REAL	실수 (소수점)	3.14, 0.5
TEXT	문자열	'김철수', 'hello'
BLOB	바이너리 데이터	이미지, 파일 등
NULL	값 없음	NULL

```sql
CREATE TABLE products (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,            -- 상품명 (문자열)
  price INTEGER NOT NULL,        -- 가격 (정수)
  weight REAL,                   -- 무게 (실수)
  description TEXT,              -- 설명 (문자열, NULL 허용)
  image BLOB                    -- 이미지 (바이너리)
);
```

---

6. Node.js에서 DB 연결

better-sqlite3: 동기 방식, 간단

`better-sqlite3`는 Node.js에서 SQLite를 사용하는 가장 간단한 방법입니다. **동기(synchronous)** 방식이라 코드가 직관적입니다.

```bash
# 설치
npm install better-sqlite3
```

```
const Database = require('better-sqlite3');

// 데이터베이스 열기 (파일이 없으면 자동 생성)
const db = new Database('myapp.db');

// 테이블 생성
db.exec(`
  CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    age INTEGER DEFAULT 0
  )
`);

// 데이터 삽입
const insert = db.prepare('INSERT INTO users (name, email, age) VALUES (?, ?, ?)'
insert.run('김철수', 'chulsoo@email.com', 25);
insert.run('이영희', 'younghee@email.com', 30);

// 데이터 조회 (여러 행)
const users = db.prepare('SELECT * FROM users').all();
console.log(users);
// [
//   { id: 1, name: '김철수', email: 'chulsoo@email.com', age: 25 },
//   { id: 2, name: '이영희', email: 'younghee@email.com', age: 30 }
// ]

// 데이터 조회 (한 행)
const user = db.prepare('SELECT * FROM users WHERE id = ?').get(1);
console.log(user);
// { id: 1, name: '김철수', email: 'chulsoo@email.com', age: 25 }

// 데이터 수정
const update = db.prepare('UPDATE users SET age = ? WHERE id = ?');
const result = update.run(26, 1);
console.log(result.changes); // 1 (수정된 행 수)

// 데이터 삭제
const remove = db.prepare('DELETE FROM users WHERE id = ?');
remove.run(2);

// 데이터베이스 닫기
db.close();
```

pg: PostgreSQL 비동기 연결

실무에서는 PostgreSQL을 많이 사용합니다. `pg` 라이브러리는 **비동기(asynchronous)** 방식입니다.

```
npm install pg
```

```
const { Pool } = require('pg');

const pool = new Pool({
  host: 'localhost',
  port: 5432,
  database: 'myapp',
  user: 'myuser',
  password: 'mypassword',
});

async function main() {
  // 데이터 조회
  const result = await pool.query('SELECT * FROM users WHERE age ≥ $1', [25]);
  console.log(result.rows);

  // 데이터 삽입
  await pool.query(
    'INSERT INTO users (name, email, age) VALUES ($1, $2, $3)',
    ['김철수', 'chulsoo@email.com', 25]
  );

  // 연결 종료
  await pool.end();
}

main();
```

Prepared Statement: SQL 인젝션 방지

왜 필요한가?

사용자 입력을 그대로 SQL에 넣으면 **SQL 인젝션** 공격에 취약합니다. 이것은 웹 보안에서 가장 위험한 공격 중 하나입니다.

실생활 비유: 은행 창구

은행에 가면 **정해진 양식**에 맞게 정보를 기입합니다. 양식 안에 "이 계좌의 모든 돈을 인출해주세요"라고 써도 은행원은 양식의 해당 칸만 읽습니다. Prepared Statement도 마찬가지로, 사용자 입력을 "데이터"로만 취급하고 "명령어"로 해석하지 않습니다.

```
// 위험한 방법 (SQL 인젝션 취약!)
const name = "'; DROP TABLE users; --";
db.exec(`SELECT * FROM users WHERE name = '${name}'`);
// 실행되는 SQL: SELECT * FROM users WHERE name = ''; DROP TABLE users; --'
// → 테이블이 삭제됩니다!

// 안전한 방법 (Prepared Statement 사용)
const stmt = db.prepare('SELECT * FROM users WHERE name = ?');
const user = stmt.get(name);
// ?에 들어가는 값은 "데이터"로만 처리됨
// → SQL 명령어로 해석되지 않아 안전합니다
```

```
// better-sqlite3에서 Prepared Statement 활용
const db = new Database('myapp.db');

// ? 를 사용한 위치 기반 파라미터
const stmt1 = db.prepare('SELECT * FROM users WHERE age ≥ ? AND age ≤ ?');
const result1 = stmt1.all(20, 30);

// @name 을 사용한 이름 기반 파라미터
const stmt2 = db.prepare('SELECT * FROM users WHERE name = @name AND age = @age')
const result2 = stmt2.all({ name: '김철수', age: 25 });

// 트랜잭션: 여러 작업을 하나로 묶기
const insertMany = db.transaction((users) => {
  const insert = db.prepare('INSERT INTO users (name, email, age) VALUES (?, ?, ?
  for (const user of users) {
    insert.run(user.name, user.email, user.age);
  }
});

insertMany([
  { name: '홍길동', email: 'hong@email.com', age: 20 },
  { name: '성춘향', email: 'chun@email.com', age: 18 },
]);
// 둘 다 성공하거나, 둘 다 실패합니다 (원자성)
```

7. Day 21 핵심 정리

개념	설명	SQL 예시
DDL	구조 정의	CREATE TABLE, DROP TABLE, ALTER TABLE
DML	데이터 조작	INSERT, SELECT, UPDATE, DELETE
PRIMARY KEY	행을 고유하게 식별	`id INTEGER PRIMARY KEY`
NOT NULL	빈 값 불허	`name TEXT NOT NULL`
Prepared Statement	SQL 인젝션 방지	`db.prepare('SELECT * FROM users WHERE id = ?')`

Day 22 - SQL 심화: 집계와 분석

1. 집계 함수

왜 필요한가?

데이터를 단순히 조회하는 것만으로는 부족합니다. "총 주문금액이 얼마인지", "평균 나이가 몇 살인지", "가장 비싼 상품은 무엇인지"와 같은 **분석**이 필요합니다. 집계 함수는 여러 행의 데이터를 하나의 결과로 요약해 줍니다.

실생활 비유: 엑셀의 SUM, AVERAGE 함수

엑셀에서 셀 범위를 선택하고 `=SUM(A1:A10)`을 쓰면 합계를 구할 수 있죠? SQL의 집계 함수도 같은 역할을 합니다.

주요 집계 함수

```sql
-- 예제 테이블: orders (주문)
CREATE TABLE orders (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  customer_name TEXT NOT NULL,
  product TEXT NOT NULL,
  amount INTEGER NOT NULL,
  order_date TEXT NOT NULL
);

INSERT INTO orders (customer_name, product, amount, order_date) VALUES
  ('김철수', '노트북', 1200000, '2026-01-15'),
  ('김철수', '마우스', 35000, '2026-01-20'),
  ('이영희', '키보드', 89000, '2026-02-01'),
  ('이영희', '모니터', 450000, '2026-02-10'),
  ('박민수', '노트북', 1350000, '2026-02-15'),
  ('박민수', '마우스', 42000, '2026-03-01'),
  ('최지은', '키보드', 95000, '2026-03-05');
```

```sql
-- COUNT: 행의 개수
SELECT COUNT(*) FROM orders;                    -- 결과: 7
SELECT COUNT(*) FROM orders WHERE product = '노트북';  -- 결과: 2

-- SUM: 합계
SELECT SUM(amount) FROM orders;                 -- 결과: 3261000
SELECT SUM(amount) FROM orders WHERE customer_name = '김철수';  -- 결과: 1235000

-- AVG: 평균
SELECT AVG(amount) FROM orders;                 -- 결과: 465857.14...
SELECT ROUND(AVG(amount), 0) FROM orders;       -- 결과: 465857 (반올림)

-- MAX: 최대값
SELECT MAX(amount) FROM orders;                 -- 결과: 1350000

-- MIN: 최소값
SELECT MIN(amount) FROM orders;                 -- 결과: 35000

-- 여러 집계 함수를 동시에 사용
SELECT
  COUNT(*) AS 주문수,
  SUM(amount) AS 총금액,
  ROUND(AVG(amount), 0) AS 평균금액,
  MAX(amount) AS 최대금액,
  MIN(amount) AS 최소금액
FROM orders;
```

---

2. GROUP BY: 그룹별 집계

왜 필요한가?

전체 합계나 평균만으로는 부족할 때가 있습니다. "고객별 총 주문금액", "상품별 판매 수량"처럼 **그룹을 나눠서** 집계해야 할 때 GROUP BY를 사용합니다.

실생활 비유: 반별 시험 평균

학교에서 전교생의 시험 평균을 구할 수도 있지만, **반별로** 평균을 구하면 더 유용한 정보를 얻을 수 있습니다. GROUP BY는 "반별로 나누어서 계산하세요"라고 말하는 것과 같습니다.

```sql
-- 고객별 주문 통계
SELECT
  customer_name AS 고객,
  COUNT(*) AS 주문횟수,
  SUM(amount) AS 총금액,
  ROUND(AVG(amount), 0) AS 평균금액
FROM orders
GROUP BY customer_name;

-- 결과:
--  ┌──────────┬────────────┬──────────┬────────────┐
-- │ 고객     │ 주문횟수   │ 총금액   │ 평균금액   │
-- ├──────────┼────────────┼──────────┼────────────┤
-- │ 김철수   │ 2          │ 1235000 │ 617500     │
-- │ 이영희   │ 2          │ 539000  │ 269500     │
-- │ 박민수   │ 2          │ 1392000 │ 696000     │
-- │ 최지은   │ 1          │ 95000   │ 95000      │
--  └──────────┴────────────┴──────────┴────────────┘
```

```sql
-- 상품별 판매 통계
SELECT
  product AS 상품,
  COUNT(*) AS 판매수,
  SUM(amount) AS 총매출
FROM orders
GROUP BY product
ORDER BY 총매출 DESC;

-- 월별 매출 통계
SELECT
  substr(order_date, 1, 7) AS 월,
  COUNT(*) AS 주문수,
  SUM(amount) AS 월매출
FROM orders
GROUP BY substr(order_date, 1, 7)
ORDER BY 월;
```

---

3. HAVING: 그룹 조건

왜 필요한가?

GROUP BY로 그룹을 만든 후, **특정 조건을 만족하는 그룹만** 보고 싶을 때 HAVING을 사용합니다.

WHERE vs HAVING 차이

이 둘의 차이를 이해하는 것이 매우 중요합니다.

구분	WHERE	HAVING
적용 시점	그룹화 **전**	그룹화 **후**
대상	개별 행	그룹 (집계 결과)
비유	학생 개인의 점수가 80점 이상인 학생	반 평균이 80점 이상인 반

```sql
-- WHERE: 개별 행을 필터링 (그룹화 전)
-- "주문금액이 100000 이상인 주문만 골라서"
SELECT customer_name, SUM(amount)
FROM orders
WHERE amount ≥ 100000
GROUP BY customer_name;

-- HAVING: 그룹을 필터링 (그룹화 후)
-- "총 주문금액이 500000 이상인 고객만"
SELECT customer_name, SUM(amount) AS total
FROM orders
GROUP BY customer_name
HAVING total ≥ 500000;

-- WHERE와 HAVING 동시 사용
-- "노트북 주문만 골라서(WHERE), 총금액이 100만 이상인 고객만(HAVING)"
SELECT customer_name, SUM(amount) AS total
FROM orders
WHERE product = '노트북'
GROUP BY customer_name
HAVING total ≥ 1000000;
```

SQL 실행 순서

SQL은 작성 순서와 실행 순서가 다릅니다. 이것을 이해하면 WHERE와 HAVING의 차이가 명확해집니다.

```
작성 순서:  SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT
실행 순서:  FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

1. FROM orders            -- 테이블에서
2. WHERE amount ≥ 100     -- 조건에 맞는 행을 필터링
3. GROUP BY customer      -- 그룹으로 묶고
4. HAVING SUM(amount) > 1000  -- 그룹 조건 필터링
5. SELECT customer, SUM   -- 결과를 선택
6. ORDER BY SUM DESC      -- 정렬
7. LIMIT 10               -- 제한
```

---

4. 서브쿼리: 쿼리 안의 쿼리

왜 필요한가?

복잡한 질문에 답하려면 한 번의 쿼리로는 부족할 때가 있습니다. "평균 주문금액보다 비싼 주문은?"처럼, 먼저 평균을 구한 다음 그 값을 이용해 필터링해야 합니다.

실생활 비유: 봉투 안에 봉투

편지 봉투를 열면 안에 또 다른 편지 봉투가 들어있는 것과 같습니다. 안쪽 봉투(서브쿼리)의 편지를 먼저 읽고, 그 내용을 바탕으로 바깥 봉투(메인 쿼리)의 편지를 이해합니다.

```sql
-- 평균 주문금액보다 비싼 주문 찾기
SELECT * FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders);
-- 안쪽 쿼리가 먼저 실행: AVG(amount) = 465857
-- 바깥 쿼리: WHERE amount > 465857

-- 가장 비싼 주문을 한 고객의 이름
SELECT customer_name FROM orders
WHERE amount = (SELECT MAX(amount) FROM orders);

-- 2번 이상 주문한 고객의 주문 내역
SELECT * FROM orders
WHERE customer_name IN (
  SELECT customer_name FROM orders
  GROUP BY customer_name
  HAVING COUNT(*) ≥ 2
);

-- FROM 절에서 서브쿼리 사용 (파생 테이블)
SELECT 고객, 총금액
FROM (
  SELECT customer_name AS 고객, SUM(amount) AS 총금액
  FROM orders
  GROUP BY customer_name
) AS customer_totals
WHERE 총금액 >= 1000000;
```

자주 하는 실수

1. **서브쿼리가 여러 행을 반환하는데 `=` 사용**: 서브쿼리가 여러 값을 반환하면 `=` 대신 `IN`을 사용하세요.

```sql
-- 오류! 서브쿼리가 여러 행을 반환
SELECT * FROM users WHERE id = (SELECT user_id FROM orders);

-- 올바른 방법
SELECT * FROM users WHERE id IN (SELECT user_id FROM orders);
```

2. **서브쿼리 남용**: 서브쿼리는 편리하지만, 때로는 JOIN이 더 효율적입니다. 성능이 중요한 경우 JOIN을 고려하세요.

---

5. VIEW: 저장된 쿼리

왜 필요한가?

자주 사용하는 복잡한 쿼리를 매번 다시 작성하는 것은 번거롭습니다. VIEW는 쿼리를 **저장**해두고 마치 테이블처럼 사용할 수 있게 해줍니다.

실생활 비유: 즐겨찾기/북마크

자주 방문하는 웹사이트의 주소를 매번 입력하는 대신, **즐겨찾기**에 저장해두고 클릭 한 번으로 접근하는 것과 같습니다.

```sql
-- VIEW 생성
CREATE VIEW customer_summary AS
SELECT
  customer_name AS 고객,
  COUNT(*) AS 주문횟수,
  SUM(amount) AS 총금액,
  ROUND(AVG(amount), 0) AS 평균금액
FROM orders
GROUP BY customer_name;

-- VIEW를 테이블처럼 사용
SELECT * FROM customer_summary;
SELECT * FROM customer_summary WHERE 총금액 >= 1000000;
SELECT * FROM customer_summary ORDER BY 주문횟수 DESC;

-- VIEW 삭제
DROP VIEW IF EXISTS customer_summary;
```

VIEW는 데이터를 직접 저장하지 않습니다. VIEW를 조회할 때마다 원본 테이블에서 데이터를 가져옵니다. 따라서 원본 데이터가 바뀌면 VIEW의 결과도 자동으로 바뀝니다.

---

6. CASE 문: 조건부 값 변환

왜 필요한가?

데이터를 조회할 때, 특정 조건에 따라 값을 변환하고 싶을 때가 있습니다. 예를 들어 주문금액에 따라 "VIP", "일반", "소액" 등으로 분류하고 싶을 때 CASE 문을 사용합니다.

```sql
-- 주문금액에 따른 등급 분류
SELECT
  customer_name,
  product,
  amount,
  CASE
    WHEN amount ≥ 1000000 THEN 'VIP 주문'
    WHEN amount ≥ 100000 THEN '일반 주문'
    ELSE '소액 주문'
  END AS 주문등급
FROM orders;

-- 결과:
-- ┌──────────────┬─────────┬──────────┬──────────┐
-- │ customer_name│ product│ amount   │ 주문등급  │
-- ├──────────────┼─────────┼──────────┼──────────┤
-- │ 김철수       │ 노트북  │ 1200000 │ VIP 주문  │
-- │ 김철수       │ 마우스  │ 35000   │ 소액 주문 │
-- │ 이영희       │ 키보드  │ 89000   │ 소액 주문 │
-- │ 이영희       │ 모니터  │ 450000  │ 일반 주문 │
-- └──────────────┴─────────┴──────────┴──────────┘

-- CASE와 집계 함수를 결합
SELECT
  customer_name,
  COUNT(CASE WHEN amount ≥ 100000 THEN 1 END) AS 큰주문수,
  COUNT(CASE WHEN amount < 100000 THEN 1 END) AS 작은주문수
FROM orders
GROUP BY customer_name;
```

---

7. 인덱스: 검색 성능 향상

왜 필요한가?

데이터가 수만, 수십만 건이 되면 검색 속도가 느려집니다. 인덱스는 데이터를 빠르게 찾을 수 있도록 도와주는 구조입니다.

실생활 비유: 책의 색인(INDEX)

500페이지짜리 책에서 "데이터베이스"라는 단어를 찾으려면 어떻게 하나요?

- **인덱스 없이**: 1페이지부터 500페이지까지 하나하나 넘기며 찾기 (Full Table Scan)
- **인덱스 있으면**: 뒤쪽의 색인(INDEX)에서 "데이터베이스 → 42p, 158p, 302p"를 보고 바로 해당 페이지로 이동

```sql
-- 인덱스 생성
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_orders_customer ON orders(customer_name);
CREATE INDEX idx_orders_date ON orders(order_date);

-- 복합 인덱스 (여러 컬럼)
CREATE INDEX idx_orders_customer_date ON orders(customer_name, order_date);

-- 고유 인덱스 (UNIQUE 제약조건과 유사)
CREATE UNIQUE INDEX idx_users_email_unique ON users(email);

-- 인덱스 삭제
DROP INDEX IF EXISTS idx_users_email;
```

EXPLAIN QUERY PLAN: 쿼리 실행 계획 확인

쿼리가 인덱스를 사용하는지 확인할 수 있습니다.

```sql
-- 인덱스 없이 검색 (느림)
EXPLAIN QUERY PLAN SELECT * FROM orders WHERE customer_name = '김철수';
-- 결과: SCAN orders  ← 전체를 훑어봄 (느림)

-- 인덱스 생성 후
CREATE INDEX idx_orders_customer ON orders(customer_name);

EXPLAIN QUERY PLAN SELECT * FROM orders WHERE customer_name = '김철수';
-- 결과: SEARCH orders USING INDEX idx_orders_customer  ← 인덱스 사용 (빠름)
```

언제 인덱스를 만드는가?

상황	인덱스 필요?	이유
WHERE 절에 자주 사용되는 컬럼	O	검색 속도 향상
JOIN에 사용되는 컬럼	O	결합 속도 향상
ORDER BY에 사용되는 컬럼	O	정렬 속도 향상
데이터가 적은 테이블	X	효과가 미미함
INSERT/UPDATE가 매우 빈번한 테이블	주의	쓰기 성능 저하
값의 종류가 적은 컬럼 (성별 등)	X	효과가 적음

자주 하는 실수

1. **인덱스를 너무 많이 만드는 것**: 인덱스는 읽기 속도를 높이지만, 쓰기(INSERT, UPDATE, DELETE) 속도는 느려집니다. 책에 색인이 너무 많으면 색인 자체를 관리하는 비용이 커지는 것과 같습니다.
2. **복합 인덱스의 순서를 잘못 정하는 것**: `CREATE INDEX idx ON orders(customer_name, order_date)`에서 `customer_name`이 먼저 오므로, `WHERE customer_name = ?`에는 이 인덱스가 사용되지만 `WHERE order_date = ?`만으로는 사용되지 않습니다.

---

8. Day 22 핵심 정리

개념	설명	SQL 예시
집계 함수	여러 행을 하나로 요약	COUNT, SUM, AVG, MAX, MIN
GROUP BY	그룹별로 집계	`GROUP BY customer_name`
HAVING	그룹 조건 필터링	`HAVING SUM(amount) > 1000`
서브쿼리	쿼리 안에 쿼리	`WHERE amount > (SELECT AVG(amount) ...)`
VIEW	저장된 쿼리	`CREATE VIEW summary AS SELECT ...`
CASE	조건부 값 변환	`CASE WHEN ... THEN ... END`
인덱스	검색 속도 향상	`CREATE INDEX idx ON table(column)`

Day 23 - 데이터 모델링: 관계와 JOIN

1. ERD (Entity-Relationship Diagram)

왜 필요한가?

실제 애플리케이션을 만들기 전에, 데이터를 어떤 테이블로 나누고 어떻게 연결할지 **설계**해야 합니다. ERD는 이 설계를 시각적으로 표현한 다이어그램입니다.

실생활 비유: 조직도/가족 관계도

회사의 조직도를 보면 부서와 직원의 관계를 한눈에 파악할 수 있습니다. ERD도 마찬가지로 테이블 간의 관계를 한눈에 보여줍니다.

```
┌──────────────┐          ┌──────────────┐
│   teachers   │          │   students   │
├──────────────┤          ├──────────────┤
│ id (PK)      │──────┐   │ id (PK)      │
│ name         │      │   │ name         │
│ subject      │      └──→│ teacher_id(FK)│
└──────────────┘          │ grade        │
                          └──────────────┘
  선생님 1명이              여러 학생을 담당
```

ERD의 주요 구성 요소:

- **엔티티(Entity)**: 테이블 (사각형으로 표현)
- **속성(Attribute)**: 컬럼 (엔티티 안에 나열)
- **관계(Relationship)**: 테이블 간의 연결 (선으로 표현)
- **PK**: Primary Key (기본 키)
- **FK**: Foreign Key (외래 키)

---

2. 1:N 관계 (일대다)

왜 필요한가?

데이터베이스에서 가장 흔한 관계입니다. "하나"가 "여러 개"와 연결되는 구조입니다.

실생활 비유: 선생님과 학생

담임 선생님 **한 명**이 여러 학생을 담당합니다. 하지만 학생은 담임 선생님이 **한 명**입니다.

- 선생님 → 학생: 1:N (한 명이 여러 명을)
- 학생 → 선생님: N:1 (여러 명이 한 명에게)

```sql
-- 선생님 테이블
CREATE TABLE teachers (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  subject TEXT NOT NULL
);

-- 학생 테이블 (선생님을 참조)
CREATE TABLE students (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  grade INTEGER NOT NULL,
  teacher_id INTEGER NOT NULL,
  FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);
```

Foreign Key (외래 키)

외래 키는 **다른 테이블의 PRIMARY KEY를 참조**하는 컬럼입니다. 이를 통해 테이블 간의 관계를 만듭니다.

```sql
-- 데이터 삽입
INSERT INTO teachers (name, subject) VALUES ('김선생', '수학');
INSERT INTO teachers (name, subject) VALUES ('이선생', '영어');

INSERT INTO students (name, grade, teacher_id) VALUES ('홍길동', 3, 1);
INSERT INTO students (name, grade, teacher_id) VALUES ('성춘향', 2, 1);
INSERT INTO students (name, grade, teacher_id) VALUES ('이몽룡', 3, 2);

-- teacher_id = 1 → teachers 테이블에서 id가 1인 '김선생'을 가리킴
-- teacher_id = 2 → teachers 테이블에서 id가 2인 '이선생'을 가리킴
```

외래 키 제약조건의 역할

```sql
-- 존재하지 않는 선생님을 참조하면? → 오류!
INSERT INTO students (name, grade, teacher_id) VALUES ('장보고', 1, 999);
-- FOREIGN KEY constraint failed (teachers 테이블에 id=999가 없음)

-- SQLite에서 외래 키 활성화 (기본적으로 비활성)
PRAGMA foreign_keys = ON;
```

다른 1:N 관계 예시

```sql
-- 게시판: 사용자(1) ↔ 게시글(N)
CREATE TABLE posts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  content TEXT NOT NULL,
  user_id INTEGER NOT NULL,
  created_at TEXT DEFAULT (datetime('now')),
  FOREIGN KEY (user_id) REFERENCES users(id)
);

-- 댓글: 게시글(1) ↔ 댓글(N)
CREATE TABLE comments (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  content TEXT NOT NULL,
  post_id INTEGER NOT NULL,
  user_id INTEGER NOT NULL,
  created_at TEXT DEFAULT (datetime('now')),
  FOREIGN KEY (post_id) REFERENCES posts(id),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

---

3. N:M 관계 (다대다)

왜 필요한가?

"한 명의 학생이 여러 수업을 듣고, 하나의 수업에 여러 학생이 있다"와 같은 관계는 1:N으로 표현할 수 없습니다.

실생활 비유: 학생과 수업

- 학생 한 명이 수학, 영어, 과학 등 **여러 수업**을 수강합니다.
- 수학 수업에는 **여러 학생**이 등록되어 있습니다.

- 이것이 N:M (다대다) 관계입니다.

중간 테이블 (Junction Table)

N:M 관계는 데이터베이스에서 직접 표현할 수 없습니다. 대신 **중간 테이블**을 만들어 두 개의 1:N 관계로 분해합니다.

```
┌──────────┐      ┌──────────────────┐      ┌──────────┐
│ students │      │ enrollments      │      │ courses  │
├──────────┤      ├──────────────────┤      ├──────────┤
│ id (PK)  │◄────►│ student_id (FK)  │      │ id (PK)  │
│ name     │      │ course_id (FK)   │◄────►│ title    │
│          │      │ enrolled_at      │      │ credits  │
└──────────┘      └──────────────────┘      └──────────┘

학생(1) ↔ 수강(N)     수강(N) ↔ 수업(1)
→ 결과적으로 학생(N) ↔ 수업(M)
```

```sql
-- 학생 테이블
CREATE TABLE students (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL
);

-- 수업 테이블
CREATE TABLE courses (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  credits INTEGER NOT NULL
);

-- 중간 테이블 (수강 등록)
CREATE TABLE enrollments (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  student_id INTEGER NOT NULL,
  course_id INTEGER NOT NULL,
  enrolled_at TEXT DEFAULT (datetime('now')),
  FOREIGN KEY (student_id) REFERENCES students(id),
  FOREIGN KEY (course_id) REFERENCES courses(id),
  UNIQUE(student_id, course_id)  -- 같은 학생이 같은 수업에 중복 등록 방지
);

-- 데이터 삽입
INSERT INTO students (name) VALUES ('홍길동'), ('성춘향'), ('이몽룡');
INSERT INTO courses (title, credits) VALUES ('수학', 3), ('영어', 3), ('과학', 2);

-- 수강 등록
INSERT INTO enrollments (student_id, course_id) VALUES (1, 1);  -- 홍길동 → 수학
INSERT INTO enrollments (student_id, course_id) VALUES (1, 2);  -- 홍길동 → 영어
INSERT INTO enrollments (student_id, course_id) VALUES (2, 1);  -- 성춘향 → 수학
INSERT INTO enrollments (student_id, course_id) VALUES (2, 3);  -- 성춘향 → 과학
INSERT INTO enrollments (student_id, course_id) VALUES (3, 2);  -- 이몽룡 → 영어
INSERT INTO enrollments (student_id, course_id) VALUES (3, 3);  -- 이몽룡 → 과학
```

다른 N:M 관계 예시

```
-- 태그 시스템: 게시글(N) ↔ 태그(M)
-- 게시글 하나에 여러 태그, 태그 하나에 여러 게시글

CREATE TABLE tags (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL UNIQUE
);

CREATE TABLE post_tags (
  post_id INTEGER NOT NULL,
  tag_id INTEGER NOT NULL,
  PRIMARY KEY (post_id, tag_id),
  FOREIGN KEY (post_id) REFERENCES posts(id),
  FOREIGN KEY (tag_id) REFERENCES tags(id)
);
```

---

4. 정규화 기초

왜 필요한가? (데이터 중복 제거)

정규화란 **데이터의 중복을 최소화**하기 위해 테이블을 적절하게 나누는 과정입니다.

비정규화 상태의 문제

```
┌──────────────────────────────────────────────────────────────────┐
│                        orders (비정규화)                          │
├────┬──────────┬──────────────┬────────┬──────────┬────────┤
│ id │ 고객이름 │ 고객연락처   │ 상품명 │ 상품가격 │ 수량   │
├────┼──────────┼──────────────┼────────┼──────────┼────────┤
│ 1  │ 김철수   │ 010-1234-5678 │ 노트북 │ 1200000  │ 1      │
│ 2  │ 김철수   │ 010-1234-5678 │ 마우스 │ 35000    │ 2      │
│ 3  │ 이영희   │ 010-9876-5432 │ 노트북 │ 1200000  │ 1      │
│ 4  │ 김철수   │ 010-1234-0000 │ 키보드 │ 89000    │ 1      │
└────┴──────────┴──────────────┴────────┴──────────┴────────┘
```

문제점:

1. **데이터 중복**: 김철수의 이름과 연락처가 여러 번 반복 저장됨
2. **수정 이상(Update Anomaly)**: 김철수의 연락처가 바뀌면 모든 행을 수정해야 함 (4번 행은 이미 다른 번호!)
3. **삭제 이상(Delete Anomaly)**: 이영희의 유일한 주문을 삭제하면 이영희 고객 정보 자체가 사라짐
4. **삽입 이상(Insert Anomaly)**: 주문 없이 고객 정보만 저장할 수 없음

정규화 단계

1NF (제1정규형): 원자값

모든 컬럼이 **하나의 값**만 가져야 합니다. 콤마로 구분된 여러 값을 넣으면 안 됩니다.

```
-- 1NF 위반 (하나의 컬럼에 여러 값)
┌────┬──────────┬──────────────────┐
│ id │ name     │ phone_numbers    │
├────┼──────────┼──────────────────┤
│ 1  │ 김철수   │ 010-1234, 010-5678 │  ← 여러 값이 하나에!
└────┴──────────┴──────────────────┘

-- 1NF 준수 (별도 테이블로 분리)
┌────┬──────────┐      ┌────┬─────────┬────────────┐
│ id │ name     │      │ id │ user_id │ phone      │
├────┼──────────┤      ├────┼─────────┼────────────┤
│ 1  │ 김철수   │      │ 1  │ 1       │ 010-1234   │
└────┴──────────┘      │ 2  │ 1       │ 010-5678   │
                       └────┴─────────┴────────────┘
```

2NF (제2정규형): 부분 종속 제거

1NF를 만족하면서, **기본 키의 일부에만 종속되는 컬럼**이 없어야 합니다.

```
-- 2NF 위반 (student_id만으로 student_name이 결정됨)
┌────────────┬──────────┬──────────────┬─────┐
│ student_id │ course_id │ student_name │ grade│
├────────────┼──────────┼──────────────┼─────┤
│ 1          │ 101      │ 홍길동       │ A   │
│ 1          │ 102      │ 홍길동       │ B   │  ← 이름 중복!
└────────────┴──────────┴──────────────┴─────┘

-- 2NF 준수 (학생 정보를 별도 테이블로)
students: (id, name)
grades: (student_id, course_id, grade)
```

3NF (제3정규형): 이행 종속 제거

2NF를 만족하면서, **기본 키가 아닌 컬럼이 다른 기본 키가 아닌 컬럼에 종속**되면 안 됩니다.

```
-- 3NF 위반 (department_name이 department_id에 종속)
┌────┬──────────┬─────────────┬──────────────┐
│ id │ name     │ department_id │ department_name  │
├────┼──────────┼─────────────┼──────────────┤
│ 1  │ 김철수   │ 10          │ 개발팀       │
│ 2  │ 이영희   │ 10          │ 개발팀       │  ← 부서명 중복!
└────┴──────────┴─────────────┴──────────────┘

-- 3NF 준수 (부서 정보를 별도 테이블로)
employees: (id, name, department_id)
departments: (id, name)
```

실생활 비유로 정리

정규화는 **정리정돈**과 같습니다:

- **1NF**: 한 서랍에 한 종류의 물건만 넣기
- **2NF**: 옷장에는 옷만, 신발장에는 신발만
- **3NF**: 재료비 영수증은 회계 서류함에, 계약서는 법무 서류함에

---

5. JOIN으로 테이블 결합

왜 필요한가?

정규화로 테이블을 나누면 데이터 중복은 줄어들지만, 원하는 정보를 얻으려면 **여러 테이블을 합쳐야** 합니다. JOIN은 관련된 테이블들을 **결합**하는 기능입니다.

실생활 비유: 짝짓기

반 아이들이 짝을 지어야 하는 상황을 생각하세요:

- **INNER JOIN**: 짝이 있는 사람만 남기기 (짝이 없으면 퇴장)
- **LEFT JOIN**: 왼쪽 줄의 모든 사람은 남기기 (짝이 없으면 혼자 서 있음)

예제 데이터 준비

```sql
-- 선생님 테이블
CREATE TABLE teachers (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  subject TEXT NOT NULL
);

-- 학생 테이블
CREATE TABLE students (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  grade INTEGER NOT NULL,
  teacher_id INTEGER,  -- NULL 허용 (담임이 없는 학생도 가능)
  FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);

INSERT INTO teachers (name, subject) VALUES
  ('김선생', '수학'),
  ('이선생', '영어'),
  ('박선생', '과학');   -- 박선생은 담당 학생이 없음

INSERT INTO students (name, grade, teacher_id) VALUES
  ('홍길동', 3, 1),      -- 김선생 담당
  ('성춘향', 2, 1),      -- 김선생 담당
  ('이몽룡', 3, 2),      -- 이선생 담당
  ('장보고', 1, NULL);  -- 담임 없음
```

INNER JOIN: 양쪽 모두 있는 것만

양쪽 테이블에서 **매칭되는 데이터만** 결과에 포함합니다.

```sql
SELECT
  students.name AS 학생이름,
  students.grade AS 학년,
  teachers.name AS 담임선생님,
  teachers.subject AS 과목
FROM students
INNER JOIN teachers ON students.teacher_id = teachers.id;

-- 결과:
-- ┌──────────┬────┬──────────┬────┐
-- │ 학생이름 │ 학년 │ 담임선생님 │ 과목 │
-- ├──────────┼────┼──────────┼────┤
-- │ 홍길동   │ 3  │ 김선생   │ 수학 │
-- │ 성춘향   │ 2  │ 김선생   │ 수학 │
-- │ 이몽룡   │ 3  │ 이선생   │ 영어 │
-- └──────────┴────┴──────────┴────┘
-- 장보고(담임 없음)와 박선생(학생 없음)은 제외됨!
```

LEFT JOIN: 왼쪽 전체 + 오른쪽 매칭

왼쪽 테이블의 **모든 데이터**를 포함하고, 오른쪽 테이블에서 매칭되는 데이터가 없으면 NULL로 채웁니다.

```sql
-- 모든 학생을 보여주되, 담임 정보도 함께 (없으면 NULL)
SELECT
  students.name AS 학생이름,
  students.grade AS 학년,
  teachers.name AS 담임선생님,
  teachers.subject AS 과목
FROM students
LEFT JOIN teachers ON students.teacher_id = teachers.id;

-- 결과:
-- ┌──────────┬────┬──────────┬────┐
-- │ 학생이름 │ 학년 │ 담임선생님 │ 과목 │
-- ├──────────┼────┼──────────┼────┤
-- │ 홍길동   │ 3  │ 김선생   │ 수학 │
-- │ 성춘향   │ 2  │ 김선생   │ 수학 │
-- │ 이몽룡   │ 3  │ 이선생   │ 영어 │
-- │ 장보고   │ 1  │ NULL     │ NULL │  ← 담임 없는 학생도 포함!
-- └──────────┴────┴──────────┴────┘
```

```sql
-- 모든 선생님을 보여주되, 담당 학생도 함께 (없으면 NULL)
SELECT
  teachers.name AS 선생님,
  teachers.subject AS 과목,
  students.name AS 담당학생
FROM teachers
LEFT JOIN students ON teachers.id = students.teacher_id;

-- 결과:
-- ┌─────────┬──────┬──────────┐
-- │ 선생님  │ 과목 │ 담당학생 │
-- ├─────────┼──────┼──────────┤
-- │ 김선생  │ 수학 │ 홍길동   │
-- │ 김선생  │ 수학 │ 성춘향   │
-- │ 이선생  │ 영어 │ 이몽룡   │
-- │ 박선생  │ 과학 │ NULL     │  ← 학생 없는 선생님도 포함!
-- └─────────┴──────┴──────────┘
```

여러 테이블 JOIN

```sql
-- 학생, 수강, 수업을 모두 결합 (N:M 관계 조회)
SELECT
  s.name AS 학생,
  c.title AS 수업,
  c.credits AS 학점,
  e.enrolled_at AS 등록일
FROM students s
INNER JOIN enrollments e ON s.id = e.student_id
INNER JOIN courses c ON e.course_id = c.id
ORDER BY s.name, c.title;

-- 결과:
-- ┌────────┬──────┬──────┬─────────────────────┐
-- │ 학생   │ 수업 │ 학점 │ 등록일              │
-- ├────────┼──────┼──────┼─────────────────────┤
-- │ 성춘향 │ 과학 │ 2    │ 2026-03-05 10:00:00 │
-- │ 성춘향 │ 수학 │ 3    │ 2026-03-05 10:00:00 │
-- │ 이몽룡 │ 과학 │ 2    │ 2026-03-05 10:00:00 │
-- │ 이몽룡 │ 영어 │ 3    │ 2026-03-05 10:00:00 │
-- │ 홍길동 │ 수학 │ 3    │ 2026-03-05 10:00:00 │
-- │ 홍길동 │ 영어 │ 3    │ 2026-03-05 10:00:00 │
-- └────────┴──────┴──────┴─────────────────────┘
```

JOIN과 집계 함수 결합

```sql
-- 학생별 수강 수업 수와 총 학점
SELECT
  s.name AS 학생,
  COUNT(c.id) AS 수강수업수,
  SUM(c.credits) AS 총학점
FROM students s
LEFT JOIN enrollments e ON s.id = e.student_id
LEFT JOIN courses c ON e.course_id = c.id
GROUP BY s.id, s.name;

-- 수업별 수강 학생 수
SELECT
  c.title AS 수업,
  COUNT(e.student_id) AS 수강인원
FROM courses c
LEFT JOIN enrollments e ON c.id = e.course_id
GROUP BY c.id, c.title
ORDER BY 수강인원 DESC;
```

테이블 별칭 (Alias)

JOIN을 사용할 때 테이블 이름이 길면 별칭을 사용합니다.

```sql
-- 별칭 없이 (길)
SELECT students.name, teachers.name
FROM students
INNER JOIN teachers ON students.teacher_id = teachers.id;

-- 별칭 사용 (간결)
SELECT s.name, t.name
FROM students s
INNER JOIN teachers t ON s.teacher_id = t.id;
```

자주 하는 실수

1. **JOIN 조건(ON)을 빼먹는 경우**: 모든 가능한 조합이 나옵니다 (카테시안 곱). 데이터가 많으면 결과가 폭발적으로 늘어납니다.

```sql
-- 위험! ON 절이 없음 → 3명 x 3명 = 9개 행이 나옴
SELECT * FROM students, teachers;

-- 올바른 방법
SELECT * FROM students
INNER JOIN teachers ON students.teacher_id = teachers.id;
```

2. **컬럼 이름이 모호한 경우**: 두 테이블에 같은 이름의 컬럼이 있으면 테이블 이름을 명시해야 합니다.

```sql
-- 오류! name이 students의 것인지 teachers의 것인지 모호
SELECT name FROM students INNER JOIN teachers ON ...;

-- 올바른 방법
SELECT students.name, teachers.name FROM students INNER JOIN teachers ON ...;
```

---

6. 마이그레이션

왜 필요한가?

개발이 진행되면서 데이터베이스 구조가 바뀌어야 할 때가 있습니다. 새로운 테이블을 추가하거나, 기존 컬럼을 수정하거나, 인덱스를 추가해야 합니다. 마이그레이션은 이런 **구조 변경을 체계적으로 관리**하는 방법입니다.

실생활 비유: 리모델링 설계도

집을 리모델링할 때 설계도를 만들죠. 마이그레이션도 마찬가지입니다:

- **up** = 변경 적용 (벽을 허물고 방을 합치는 설계도)
- **down** = 변경 되돌리기 (다시 벽을 세우는 설계도)
- **버전 관리** = 몇 번째 리모델링인지 기록

마이그레이션 파일 구조

```
migrations/
  001_create_users.sql
  002_create_posts.sql
  003_add_phone_to_users.sql
  004_create_comments.sql
```

각 파일에는 up(적용)과 down(되돌리기) SQL이 포함됩니다:

```sql
-- 001_create_users.sql

-- Up (적용)
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  created_at TEXT DEFAULT (datetime('now'))
);

-- Down (되돌리기)
DROP TABLE IF EXISTS users;
```

```sql
-- 003_add_phone_to_users.sql

-- Up (적용)
ALTER TABLE users ADD COLUMN phone TEXT;

-- Down (되돌리기)
-- SQLite는 컬럼 삭제를 직접 지원하지 않으므로
-- 새 테이블을 만들고 데이터를 옮기는 방식 사용
CREATE TABLE users_backup AS SELECT id, name, email, created_at FROM users;
DROP TABLE users;
ALTER TABLE users_backup RENAME TO users;
```

Node.js에서 간단한 마이그레이션 구현

```
const Database = require('better-sqlite3');
const fs = require('fs');
const path = require('path');

const db = new Database('myapp.db');

// 마이그레이션 이력 테이블
db.exec(`
  CREATE TABLE IF NOT EXISTS migrations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE,
    applied_at TEXT DEFAULT (datetime('now'))
  )
`);

// 마이그레이션 실행
function runMigrations() {
  const migrationsDir = path.join(__dirname, 'migrations');
  const files = fs.readdirSync(migrationsDir).sort();

  for (const file of files) {
    // 이미 적용된 마이그레이션인지 확인
    const applied = db.prepare(
      'SELECT * FROM migrations WHERE name = ?'
    ).get(file);

    if (applied) {
      console.log(`건너뜀: ${file} (이미 적용됨)`);
      continue;
    }

    // SQL 파일 읽기 및 실행
    const sql = fs.readFileSync(
      path.join(migrationsDir, file), 'utf-8'
    );

    console.log(`적용 중: ${file}`);
    db.exec(sql);

    // 적용 이력 기록
    db.prepare('INSERT INTO migrations (name) VALUES (?)').run(file);
    console.log(`완료: ${file}`);
  }
}

runMigrations();
db.close();
```

버전 관리

마이그레이션 파일은 **Git에 함께 커밋**합니다. 이렇게 하면:

- 팀원 모두가 같은 데이터베이스 구조를 유지할 수 있습니다
- 데이터베이스 구조의 변경 이력을 추적할 수 있습니다
- 문제가 생기면 이전 버전으로 돌아갈 수 있습니다

```
git add migrations/005_add_index_to_posts.sql
git commit -m "마이그레이션: posts 테이블에 인덱스 추가"
```

자주 하는 실수

1. **마이그레이션 파일을 수정하는 것**: 이미 적용된 마이그레이션 파일은 절대 수정하면 안 됩니다. 변경이 필요하면 **새로운 마이그레이션 파일**을 만드세요.
2. **down 마이그레이션을 만들지 않는 것**: 배포 후 문제가 생겼을 때 되돌릴 수 없습니다. 항상 up과 down을 함께 작성하세요.
3. **순서를 지키지 않는 것**: 마이그레이션 파일은 반드시 **번호 순서**대로 실행되어야 합니다. 번호를 건너뛰거나 순서를 바꾸면 오류가 발생합니다.

---

7. Day 23 핵심 정리

개념	설명	핵심 포인트
ERD	테이블 관계 다이어그램	설계의 시작점
1:N 관계	하나가 여러 개와 연결	Foreign Key 사용
N:M 관계	다대다 연결	중간 테이블로 분해
정규화	데이터 중복 제거	1NF → 2NF → 3NF
INNER JOIN	매칭되는 것만 결합	짝 있는 사람만
LEFT JOIN	왼쪽 전체 + 오른쪽 매칭	짝 없어도 포함
마이그레이션	DB 구조 변경 관리	up/down, 버전 관리

Phase 5 종합 정리

3일간의 학습을 통해 데이터베이스의 기초를 다졌습니다. 핵심을 정리하면:

Day 21: SQL의 기본 문법(DDL, DML)을 배우고, Node.js에서 데이터베이스를 연결하는 방법을 익혔습니다. Prepared Statement로 SQL 인젝션을 방지하는 것이 중요합니다.

Day 22: 집계 함수와 GROUP BY로 데이터를 분석하고, 서브쿼리와 VIEW로 복잡한 조회를 처리하며, 인덱스로 성능을 최적화하는 방법을 배웠습니다.

Day 23: 테이블 간의 관계(1:N, N:M)를 이해하고, JOIN으로 데이터를 결합하며, 정규화와 마이그레이션으로 데이터베이스를 체계적으로 관리하는 방법을 익혔습니다.

다음 단계인 Phase 6에서는 Express.js를 사용해 실제 백엔드 서버를 구축하고, 이 데이터베이스 지식을 활용하여 API를 개발하게 됩니다.